Instituto Politécnico Nacional

Escuela Superior de Cómputo

Unidad Académica:

Sistemas Distribuidos

Profesor:

Carreto Arellano Chadwick

Práctica 01: "Multiprocesamiento"

Alumna:

Maldonado Bedolla Daniela del Carmen

Grupo:

7CM1

13 de febrero de 2026

Índice

INTRODUCCIÓN **3**

DESARROLLO **4**

Problemática 4

Propuesta de solución 4

Implementación 5

EJECUCIÓN Y RESULTADOS **8**

CONCLUSIONES **10**

ANEXO **11**

Código 11

INTRODUCCIÓN

El multiprocesamiento es una técnica de la computación que permite ejecutar múltiples tareas de manera concurrente con el objetivo de mejorar el rendimiento y la eficiencia de un sistema. Este enfoque puede implementarse mediante procesos o mediante hilos de ejecución, dependiendo de las necesidades del programa y del nivel de aislamiento requerido.

Un **proceso** es una instancia independiente de un programa en ejecución que cuenta con su propio espacio de memoria y recursos asignados. Debido a este aislamiento, los procesos no comparten memoria directamente, lo que aumenta la seguridad y estabilidad, pero también implica un mayor consumo de recursos y mecanismos adicionales para la comunicación entre ellos.

En contraste, un **hilo** es una unidad de ejecución más ligera que existe dentro de un proceso. Los hilos comparten el mismo espacio de memoria y recursos del proceso al que pertenecen, lo que permite una comunicación más rápida entre ellos y una creación más eficiente en términos de consumo de recursos. Sin embargo, al compartir memoria, es necesario implementar mecanismos de sincronización para evitar conflictos cuando múltiples hilos acceden simultáneamente a los mismos datos.

La principal diferencia entre procesos e hilos radica en el nivel de independencia y en el costo de gestión. Mientras que los procesos ofrecen mayor aislamiento, los hilos permiten mayor eficiencia y rapidez en entornos donde se requiere interacción constante entre tareas concurrentes.

En esta práctica se implementa un problema que requiere la ejecución concurrente de múltiples tareas sobre recursos compartidos, con el objetivo de observar el comportamiento del sistema bajo condiciones simultáneas de acceso. A partir de este planteamiento, se desarrolla una solución que permite gestionar adecuadamente la concurrencia y garantizar la consistencia de los datos durante la ejecución.

Desarrollo

Problemática

El problema está delimitado a un sistema de venta concurrente con 1,000 asientos (25 filas x 40 columnas) y 2,000 compradores, donde cada comprador puede adquirir solo un boleto. Todos los compradores intentan comprar de forma simultánea y repetitiva hasta que se agoten los asientos.

La dificultad central es la concurrencia sobre recursos compartidos. Existen dos condiciones de carrera críticas:

- **Carrera sobre el estado del asiento:** dos o más compradores pueden intentar vender el mismo asiento al mismo tiempo.
- **Carrera sobre el conteo global:** múltiples compradores pueden incrementar simultáneamente el total de vendidos, generando conteos incorrectos.

Además, al existir más compradores que asientos, en la etapa final aumentan los intentos fallidos y la presión de acceso concurrente, lo que exige mantener consistencia hasta el cierre total de la venta.

Propuesta de solución

La solución se basa en un diseño concurrente con control explícito de acceso a recursos compartidos. Se ejecutan 2,000 hilos compradores, cada uno intentando adquirir un único asiento hasta lograrlo o hasta que la venta concluya.

Cada asiento se modela como un recurso crítico protegido mediante un mecanismo de exclusión mutua, de modo que antes de consultar o modificar su estado, el hilo debe adquirir su respectivo bloqueo, evitando así la doble venta. El total de boletos vendidos se gestiona mediante una variable global protegida dentro de una sección crítica, garantizando que los incrementos sean consistentes.

La finalización del sistema se controla mediante una señal global que se activa cuando se alcanza el total de 1,000 asientos vendidos. A partir de ese momento, los hilos dejan de intentar compras y concluyen su ejecución de forma ordenada.

Con esta estrategia se garantiza la integridad de los datos bajo alta concurrencia: ningún asiento puede venderse más de una vez, el conteo final es correcto y la ejecución termina sin procesos residuales.

Implementación

El programa modela un foro como una matriz de **25 filas x 40 columnas**, para un total de **1,000 asientos**. Cada posición de la matriz representa un asiento único y su estado se guarda como un valor booleano: disponible o vendido. Esta estructura permite ubicar y actualizar asientos de forma directa por coordenadas (fila, columna).

```
FILAS = 25
COLUMNAS = 40
TOTAL_ASIENTOS = FILAS * COLUMNAS
COMPRADORES_POR_ASIENTO = 2
TOTAL_COMPRADORES = TOTAL_ASIENTOS * COMPRADORES_POR_ASIENTO
```

- *TOTAL_ASIENTOS: capacidad total del foro.*
- *TOTAL_COMPRADORES: demanda total concurrente (2x capacidad).*

Modelo de asientos y estados

- Se usa una estructura bidimensional para el **estado de asientos**.
- Estado inicial: todos los asientos en disponible.
- Cuando un comprador concreta compra, el estado cambia a vendido y ya no vuelve a disponible.
- En interfaz, ese cambio se refleja visualmente (gris → azul), manteniendo coherencia entre estado lógico y representación.

```
estado_asientos = [[False for _ in range(COLUMNAS)] for _ in range(FILAS)]
locks_asientos = [[threading.Lock() for _ in range(COLUMNAS)] for _ in range(FILAS)]
```

- *False: asiento disponible.*
- *True: asiento vendido.*
- *locks_asientos[f][c]: evita que dos hilos modifiquen el mismo asiento a la vez.*

Condiciones de carrera identificadas

El diseño considera dos carreras críticas:

- **Carrera por asiento**: dos hilos podrían intentar vender el mismo asiento al mismo tiempo.

- **Carrera por conteo global**: varios hilos podrían incrementar simultáneamente el total vendido y producir un número incorrecto.

Estas carreras aparecen porque hay miles de operaciones concurrentes sobre recursos compartidos.

```
asientos_vendidos = 0
lock_asientos_vendidos = threading.Lock()
lock_terminal = threading.Lock()
```

- *asientos_vendidos: recurso compartido global.*
- *lock_asientos_vendidos: serializa incrementos del contador.*
- *venta_finalizada: señal global para detener intentos.*

Estrategia de sincronización

Para mantener consistencia se aplican dos niveles de exclusión mutua:

- **Lock por asiento**: cada asiento tiene su propio bloqueo; ningún hilo puede leer/modificar ese asiento sin adquirir primero su lock.
- **Lock para contador global**: el incremento del total de vendidos ocurre en sección crítica protegida.

```
        lock = locks_asientos[row][col]
        acquired = lock.acquire(timeout=0.01)
        if not acquired:
            continue

        try:
            if estado_asientos[row][col]:
                continue

            with lock_asientos_vendidos:
                if asientos_vendidos >= TOTAL_ASIENTOS:
                    venta_finalizada.set()
                    return

                estado_asientos[row][col] = True
                asientos_ve (variable) asientos_vendidos: int
                sold_now = asientos_vendidos

            ui_updates.put((row, col, buyer_id, sold_now))

            if sold_now >= TOTAL_ASIENTOS:
                venta_finalizada.set()

            return
        finally:
            lock.release()
```

- *Primero se protege el asiento específico.*
- *Luego se protege el contador global para evitar inconsistencias.*

Con eso se garantiza que:

- un asiento no se venda dos veces,

- el conteo global sea exacto,
- la finalización se active justo cuando se llega al total de capacidad.

Hilos de compradores

- Se crean **2,000 hilos compradores** (2 por cada asiento).
- Cada hilo representa un comprador independiente y puede comprar **máximo un boleto**.
- Cada hilo intenta repetidamente sobre asientos aleatorios con un pequeño retraso entre intentos.
- Si encuentra un asiento disponible y logra el bloqueo correspondiente, confirma compra y termina.
- Si la venta ya terminó, el hilo deja de intentar y finaliza.

```python
def start_sales():
    with lock_terminal:
        print("Bienvenido a la compra de boletos del Foro Dani")
        print(f"Número de asientos disponibles: {TOTAL_ASIENTOS}")
        print(f"Número de compradores registrados: {TOTAL_COMPRADORES}")
        print("Actualizaciones de venta:")

    for buyer in range(1, TOTAL_COMPRADORES + 1):
        thread = threading.Thread(target=attempt_purchase, args=(buyer,), daemon=False)
        threads.append(thread)
        thread.start()
        time.sleep(0.0005)
```

```python
def attempt_purchase(buyer_id):
    global asientos_vendidos

    while not venta_finalizada.is_set():
        time.sleep(random.uniform(0.1, 2.0))

        row = random.randint(0, FILAS - 1)
        col = random.randint(0, COLUMNAS - 1)
```

- *En estas secciones, los compradores hacen intentos sucesivos de forma aleatoria en todos los asientos, comprobando a las malas si ya fueron vendidos o si aún están libres.*

Control de finalización

La venta concluye al venderse los 1,000 asientos. En ese momento se activa una señal global de fin, se detienen intentos adicionales y se hace cierre ordenado de hilos (join) para evitar hilos residuales. Esto asegura que no queden procesos "zombie" y que el estado final sea consistente en lógica, GUI y terminal.

Cuando se agotan asientos, se cierra de forma ordenada.

```python
    for thread in threads:
        thread.join()
```

```python
    if sold_now >= TOTAL_ASIENTOS:
        venta_finalizada.set()

    return
```

- join() garantiza que no queden hilos activos (sin zombies).
- venta_finalizada corta intentos de compra restantes.

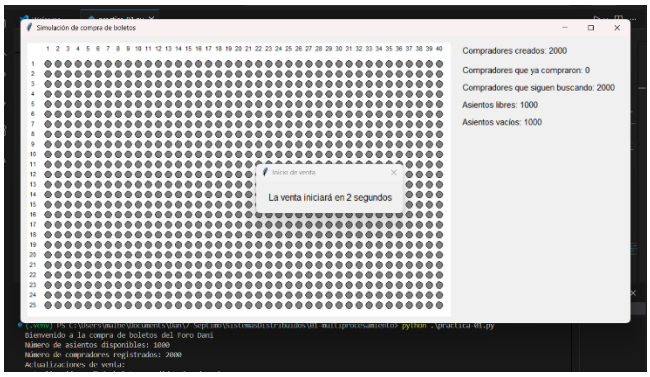
Ejecución y Resultados

Al ejecutar el programa, primero se inicializa la configuración del foro (25x40), se calculan los asientos totales y se prepara la creación de compradores concurrentes. En la terminal aparecen los mensajes de arranque: bienvenida al sistema, número de asientos disponibles y número de compradores registrados. Con esto se confirma que el entorno está listo para iniciar la venta.

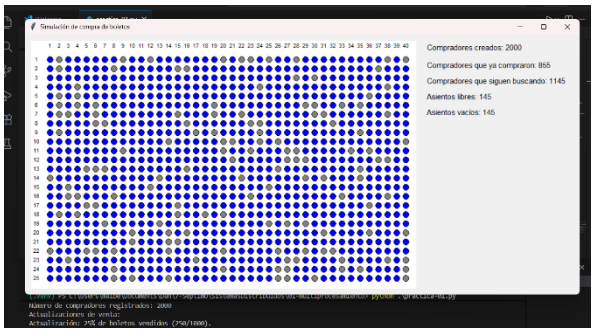
```
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\01-multiprocesamiento> python .\practica-01.py
Bienvenido a la compra de boletos del Foro Dani
Número de asientos disponibles: 1000
Número de compradores registrados: 2000
```

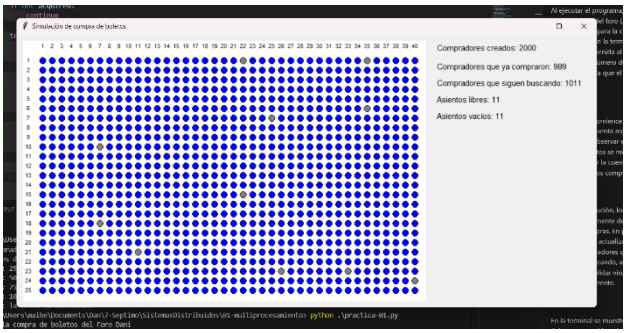
Nota: los usuarios "registrados" en sí son los hilos que van a simular ser los compradores de los asientos.

Antes de que comience la compra, aparece un pop up centrado con cuenta regresiva de 3 segundos. Este paso permite observar el estado inicial del foro, donde todos los asientos se muestran disponibles en color gris. Al terminar la cuenta regresiva, el sistema libera el inicio de los hilos compradores y arranca la venta.



Durante la ejecución, los asientos se van vendiendo y cambian de gris a azul conforme cambia su estado ha vendido. En paralelo, los contadores se actualizan: compradores que ya compraron, compradores que siguen buscando, asientos libres y asientos vacíos.

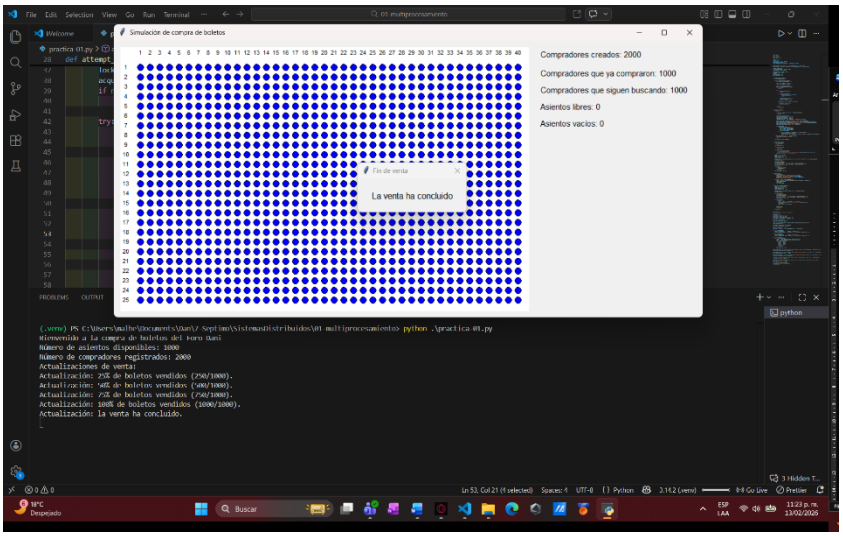




En la terminal se muestran mensajes de avance únicamente en hitos de venta: 25%, 50%, 75% y 100%, manteniendo una salida limpia y fácil de seguir. Estos porcentajes reflejan el progreso real del contador global de asientos vendidos y permiten verificar que el estado del sistema avanza de manera consistente.

```
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\01-multiprocesamiento> python .\practica-01.py
Bienvenido a la compra de boletos del Foro Dani
Número de asientos disponibles: 1000
Número de compradores registrados: 2000
Actualizaciones de venta:
Actualización: 25% de boletos vendidos (250/1000).
Actualización: 50% de boletos vendidos (500/1000).
Actualización: 75% de boletos vendidos (750/1000).
Actualización: 100% de boletos vendidos (1000/1000).
Actualización: la venta ha concluido.
(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\01-multiprocesamiento>
```

Cuando se alcanza el 100% de boletos vendidos, la venta se marca como concluida, los hilos finalizan de forma ordenada y aparece un pop up final con el mensaje “La venta ha concluido” durante 3 segundos. Después de ese tiempo, la aplicación se cierra automáticamente, completando el ciclo de ejecución sin dejar procesos activos.



Conclusiones

Después de desarrollar este problema, tengo una comprensión mucho más clara de cómo funcionan los procesos simultáneos en la práctica. Aunque procesos e hilos no son lo mismo, trabajar con hilos me permitió entender de forma concreta cómo se comportan múltiples entidades ejecutándose al mismo tiempo y compitiendo por los mismos recursos. Ahí fue clara la importancia de la sincronización y del control de acceso.

En este caso, el problema fue relativamente simple: se trataba principalmente de resolver condiciones de carrera y asegurar que la información se mantuviera consistente. Sin embargo, quedó claro que, sin mecanismos adecuados de exclusión mutua y control global, el sistema fácilmente podría volverse inconsistente.

Lo interesante es que aquí apenas se ve una parte del verdadero potencial del multiprocesamiento. La sincronización es solo el inicio. La escalabilidad que podría aplicar para prácticas futuras es donde tengo interés: cuando además de vender un asiento se deben procesar pagos o información del usuario, permitir la compra de múltiples lugares, generar comprobantes, manejar formatos de boletos o incluso coordinar varios foros desde distintos servidores. Aunque planeo exponer mis dudas de como escalar este problema en clase, este me ayudó a entender cómo dividir recursos y evitar condiciones de carrera.

En conclusión, más allá de resolver un problema puntual, esta práctica me permitió entender que la concurrencia no se trata solo de ejecutar cosas al mismo tiempo, sino de hacerlo de forma organizada, consistente y preparada para escalar.

Anexo

Código

```python
import queue
import random
import threading
import time
import tkinter as tk

FILAS = 25
COLUMNAS = 40
TOTAL_ASIENTOS = FILAS * COLUMNAS
COMPRADORES_POR_ASIENTO = 2
TOTAL_COMPRADORES = TOTAL_ASIENTOS * COMPRADORES_POR_ASIENTO


estado_asientos = [[False for _ in range(COLUMNAS)] for _ in range(FILAS)]
locks_asientos = [[threading.Lock() for _ in range(COLUMNAS)] for _ in range(FILAS)]

asientos_vendidos = 0
lock_asientos_vendidos = threading.Lock()
lock_terminal = threading.Lock()

venta_finalizada = threading.Event()
ui_updates = queue.Queue()
threads = []
cierre_iniciado = False
hilos_reportados = set()


def attempt_purchase(buyer_id):
    global asientos_vendidos

    while not venta_finalizada.is_set():
        time.sleep(random.uniform(0.1, 2.0))

        row = random.randint(0, FILAS - 1)
        col = random.randint(0, COLUMNAS - 1)

        lock = locks_asientos[row][col]
        acquired = lock.acquire(timeout=0.01)
        if not acquired:
            continue

        try:
            if estado_asientos[row][col]:
                continue

            with lock_asientos_vendidos:
                if asientos_vendidos >= TOTAL_ASIENTOS:
                    venta_finalizada.set()
                    return

                estado_asientos[row][col] = True
                asientos_vendidos += 1
                sold_now = asientos_vendidos

            ui_updates.put((row, col, buyer_id, sold_now))

            if sold_now >= TOTAL_ASIENTOS:
                venta_finalizada.set()

            return
        finally:
            lock.release()


def process_ui_updates():
    while True:
        try:
            row, col, _buyer_id, _sold_now = ui_updates.get_nowait()
        except queue.Empty:
            break

        seat_id = seat_items[row][col]
        canvas.itemconfig(seat_id, fill="blue")
        remaining = TOTAL_ASIENTOS - _sold_now
        compradores_que_compraron = _sold_now
        compradores_buscando = TOTAL_COMPRADORES - compradores_que_compraron

        buyers_total_label.config(text=f"Compradores creados: {TOTAL_COMPRADORES}")
        buyers_bought_label.config(text=f"Compradores que ya compraron: {compradores_que_compraron}")
        buyers_searching_label.config(text=f"Compradores que siguen buscando: {compradores_buscando}")
        free_label.config(text=f"Asientos libres: {remaining}")
        empty_label.config(text=f"Asientos vacios: {remaining}")

        with lock_terminal:
            for porcentaje, umbral in (
                (25, int(TOTAL_ASIENTOS * 0.25)),
                (50, int(TOTAL_ASIENTOS * 0.50)),
                (75, int(TOTAL_ASIENTOS * 0.75)),
                (100, TOTAL_ASIENTOS),
            ):
                if _sold_now >= umbral and porcentaje not in hilos_reportados:
                    hilos_reportados.add(porcentaje)
                    print(f"Actualización: {porcentaje}% de boletos vendidos ({ sold now}/{TOTAL_ASIENTOS}).")

    if venta_finalizada.is_set() and ui_updates.empty():
        root.after(150, finalize_and_close)
        return

    root.after(30, process_ui_updates)
```

```python
def finalize_and_close():
    global cierre_iniciado
    if cierre_iniciado:
        return
    cierre_iniciado = True

    for thread in threads:
        thread.join()

    with lock_terminal:
        if 100 not in hilos_reportados:
            hilos_reportados.add(100)
            print(f"Actualización: 100% de boletos vendidos ({asientos_vendidos}/{TOTAL_ASIENTOS}).")
        print("Actualización: la venta ha concluido.")

    popup = tk.Toplevel(root)
    popup.title("Fin de venta")
    popup.transient(root)
    popup.resizable(False, False)

    message_label = tk.Label(popup, text="La venta ha concluido", font=("Arial", 12), padx=24, pady=20)
    message_label.pack()

    root.update_idletasks()
    popup.update_idletasks()
    x = root.winfo_rootx() + (root.winfo_width() - popup.winfo_width()) // 2
    y = root.winfo_rootx() + (root.winfo_height() - popup.winfo_height()) // 2
    popup.geometry(f"+{x}+{y}")

    def close_all():
        if popup.winfo_exists():
            popup.destroy()
        root.destroy()

    root.after(3000, close_all)


def start_sales():
    with lock_terminal:
        print("Bienvenido a la compra de boletos del Foro Dual")
        print(f"Número de asientos disponibles: {TOTAL_ASIENTOS}")
        print(f"Número de compradores registrados: {TOTAL_COMPRADORES}")
        print("Actualizaciones de venta:")

    for buyer in range(1, TOTAL_COMPRADORES + 1):
        thread = threading.Thread(target=attempt_purchase, args=(buyer,), daemon=False)
        threads.append(thread)
        thread.start()
        time.sleep(0.0005)


def show_start_countdown(seconds=3):
    popup = tk.Toplevel(root)
    popup.title("Inicio de venta")
    popup.transient(root)
    popup.resizable(False, False)

    message_label = tk.Label(popup, font=("Arial", 12), padx=20, pady=18)
    message_label.pack()

    def center_popup():
        root.update_idletasks()
        popup.update_idletasks()
        x = root.winfo_rootx() + (root.winfo_width() - popup.winfo_width()) // 2
        y = root.winfo_rootx() + (root.winfo_height() - popup.winfo_height()) // 2
        popup.geometry(f"+{x}+{y}")

    def tick(remaining):
        if remaining > 0:
            suffix = "segundo" if remaining == 1 else "segundos"
            message_label.config(text=f"La venta iniciará en {remaining} {suffix}")
            center_popup()
            root.after(1000, tick, remaining - 1)
            return

        popup.destroy()
        start_sales()

    tick(seconds)


root = tk.Tk()
root.title("Simulación de compra de boletos")

label_margin = 30
cell_size = 18
grid_width = COLUMNAS * cell_size
grid_height = FILAS * cell_size
canvas_width = label_margin + grid_width + 10
canvas_height = label_margin + grid_height + 10

main_frame = tk.Frame(root)
main_frame.pack(padx=10, pady=10)

canvas = tk.Canvas(main_frame, width=canvas_width, height=canvas_height, bg="white")
canvas.pack(side="left")

info_frame = tk.Frame(main_frame, padx=16)
info_frame.pack(side="left", fill="y")
```

```python
radius = int(cell_size * 0.35)
seat_items = [[None for _ in range(COLUMNAS)] for _ in range(FILAS)]

for c in range(COLUMNAS):
    label_x = label_margin + c * cell_size + cell_size // 2
    canvas.create_text(label_x, 12, text=str(c + 1), fill="black", font=("Arial", 8))

for r in range(FILAS):
    label_y = label_margin + r * cell_size + cell_size // 2
    canvas.create_text(12, label_y, text=str(r + 1), fill="black", font=("Arial", 8))

for r in range(FILAS):
    for c in range(COLUMNAS):
        center_x = label_margin + c * cell_size + cell_size // 2
        center_y = label_margin + r * cell_size + cell_size // 2
        seat = canvas.create_oval(
            center_x - radius,
            center_y - radius,
            center_x + radius,
            center_y + radius,
            fill="gray",
            outline="black",
        )
        seat_items[r][c] = seat

buyers_total_label = tk.Label(info_frame, text=f"Compradores creados: {TOTAL_COMPRADORES}", font=("Arial", 11), anchor="w")
buyers_total_label.pack(fill="x", pady=(4, 8))

buyers_bought_label = tk.Label(info_frame, text="Compradores que ya compraron: 0", font=("Arial", 11), anchor="w")
buyers_bought_label.pack(fill="x", pady=4)

buyers_searching_label = tk.Label(info_frame, text=f"Compradores que siguen buscando: {TOTAL_COMPRADORES}", font=("Arial", 11), anchor="w")
buyers_searching_label.pack(fill="x", pady=4)

free_label = tk.Label(info_frame, text=f"Asientos libres: {TOTAL_ASIENTOS}", font=("Arial", 11), anchor="w")
free_label.pack(fill="x", pady=4)

empty_label = tk.Label(info_frame, text=f"Asientos vacios: {TOTAL_ASIENTOS}", font=("Arial", 11), anchor="w")
empty_label.pack(fill="x", pady=4)

root.after(10, process_ui_updates)
root.after(100, show_start_countdown, 3)
root.mainloop()
```